Sender I/O: A Constructed Comparison

Document Number:	D4053R0
Date:	2026-03-13
Audience:	LEWG

Reply-to:	Vinnie Falco vinnie.falco@gmail.com Steve Gerbino steve@gerbino.co

Table of Contents

Abstract

Revision History

- R0: March 2026 (post-Croydon mailing)

1. Disclosure
2. The Coroutine-Native Echo Server
3. Why I/O Results Are Different
4. Constructing the Sender Echo Server
5. Approach A1: Route Through `set_value`
   - What Approach A1 Loses
6. Approach A2: Split Across Channels and Shared State
   - What Approach A2 Loses
7. Approach B: Route Through `set_error`
   - What Approach B Loses
8. The Trade-Off
9. How Coroutines Achieve Structured Concurrency
10. Invitation
11. Acknowledgments

References

Abstract

Three sender-based TCP echo servers are constructed from the C++26 specification and compared against a coroutine-native echo server (Corosio[3]). Approach A1 routes I/O results through `set_value`, matching coroutine-native ergonomics but bypassing the channel-based composition algebra. Approach A2 routes the error code through `set_error` while capturing the byte count in shared state, restoring channel-based composition but removing the byte count from the completion signature and reintroducing shared mutable state. Approach B routes errors through `set_error`, retaining composition but converting every routine network event into an exception and discarding the byte count. No construction achieves both data preservation and channel-based composition without shared state or exceptions. The authors invite any reader to construct a sender-based echo server that preserves compound I/O results, retains channel-based composition, and avoids exception round-trips, using C++26 facilities. The authors will incorporate any such construction into a future revision and re-evaluate every finding.

Revision History

R0: March 2026 (post-Croydon mailing)

- Initial version.

---

1. Disclosure

The authors developed and maintain Corosio[3] and Capy[4] and believe coroutine-native I/O is the correct foundation for networking in C++. The findings in this paper are structural and hold regardless of which library implements the coroutine-native layer. This paper is one of a suite of four that examines the relationship between compound I/O results and the sender three-channel model. The authors provide information, ask nothing, and serve at the pleasure of the chair. This paper constructs the best sender-based echo server the authors can build from the C++26 specification, places it next to a coroutine-native echo server, and shows what each approach costs. The authors invite improvements to the sender construction (Section 10).

---

2. The Coroutine-Native Echo Server

Corosio[3] `do_session`:

```
capy::task<> do_session()
{
    for (;;)
    {
        auto [ec, n] = co_await sock_.read_some(
            capy::mutable_buffer(
                buf_, sizeof buf_));

        auto [wec, wn] = co_await capy::write(
            sock_, capy::const_buffer(buf_, n));

        if (wec || ec)
            break;
    }
    sock_.close();
}
```

Each `co_await` yields `(error_code, size_t)` as a structured binding. The error code is a value. No exceptions. Both values are visible at the call site.

---

3. Why I/O Results Are Different

Infrastructure operations have binary outcomes:

Operation	Success	Failure
`malloc`	Block returned	Allocation failed
`fopen`	File handle returned	Open failed
`pthread_create`	Thread running	Creation failed
GPU kernel launch	Kernel running	Launch failed
Timer arm	Timer armed	Resource limit

Every row is binary. The three channels - `set_value`, `set_error`, `set_stopped` - map unambiguously. The three-channel model is correct for this class. `std::execution` serves it well.

I/O operations return compound results:

Operation	Result
`read`	`(status, bytes_transferred)`
`write`	`(status, bytes_written)`

Status and data, always paired. Both values are always present. Every OS delivers them together. io_uring carries `res` and `flags` in one CQE. IOCP returns a `BOOL` and `lpNumberOfBytesTransferred` in one call. POSIX `read()` returns `ssize_t` with `errno` set.

Chris Kohlhoff identified this in P2430R0[5] ("Partial success scenarios with P2300," 2021):

> *"Due to the limitations of the `set_error` channel (which has a single 'error' argument) and `set_done` channel (which takes no arguments), partial results must be communicated down the `set_value` channel."*

The remaining sections construct three approaches to routing compound results through three channels and show what each costs.

---

4. Constructing the Sender Echo Server

Three approaches exist for routing I/O results through the three-channel model. Approach A1 routes everything through `set_value`. Approach A2 routes the error code through `set_error` while capturing the byte count in shared state. Approach B routes errors through `set_error` and discards the byte count. We construct the best version of each from the C++26 specification (P2300R10[1], P3552R3[2]).

Dietmar Kühl enumerated five channel-routing options for I/O in P2762R2[10] (Section 4.2), including fused `set_value(error_code, n)`, multiple `set_value` overloads, `set_error` routing, hybrid severity-based classification, and an `error_code` reference argument. Kühl noted that "some of the error cases may have been partial successes" and that "using the set_error channel taking just one argument is somewhat limiting." Kirk Shoop identified the same heuristic difficulty in P2471R1[11], observing that completion tokens translating to senders "must use a heuristic to type-match the first arg" to distinguish errors from values. The approaches below correspond to Kühl's options 1, 3, and a shared-state variant not in his enumeration.

---

5. Approach A1: Route Through `set_value`

The I/O sender calls `set_value(error_code, size_t)` for all outcomes. Completion signature: `set_value_t(std::error_code, std::size_t)`.

P3552R3[2] Section 4.4 specifies that `co_await` on a sender with multiple value arguments yields a `std::tuple`. Structured bindings work:

```
auto do_session(auto& sock, auto& buf)
    -> std::execution::task<void>
{
    for (;;)
    {
        auto [ec, n] = co_await async_read(
            sock, net::buffer(buf));

        auto [wec, wn] = co_await async_write(
            sock, net::const_buffer(buf, n));

        if (wec || ec)
            break;
    }
}
```

The echo session is nearly identical to Corosio. The error code is a value. No exceptions. Both values are visible at the call site.

What Approach A1 Loses

The I/O sender never calls `set_error` for I/O results. The channel-based composition algebra is bypassed:

- **`when_all`** cancels sibling operations when one completes with `set_error` or `set_stopped` ([exec.when.all]). An I/O failure arriving through `set_value` looks like success to `when_all`. Siblings continue.
- **`upon_error`** intercepts `set_error` ([exec.then]). It is unreachable for these senders.
- **`retry`** (stdexec, not in the C++26 working draft) intercepts `set_error` and restarts the operation. An I/O failure arriving through `set_value` does not trigger it.

The three-channel model reduces to one channel with a structured result. `set_error` serves no purpose for I/O senders under Approach A1. (`set_stopped` retains its cancellation role.)

6. Approach A2: Split Across Channels and Shared State

A variation of Approach A1 routes the error code through `set_error` while capturing the byte count into shared state before the channel split. A `let_error` handler recovers both values:

```
auto do_session(auto& sock, auto& buf)
    -> std::execution::task<void>
{
    for (;;)
    {
        std::size_t n = 0;

        co_await (
            async_read(sock, net::buffer(buf))
            | then([&](std::size_t bytes) {
                  n = bytes;
              })
            | upon_error(
                  [&](std::error_code ec) {
                      // ec visible, n stale
                  }));

        co_await (
            async_write(
                sock, net::const_buffer(buf, n))
            | then([&](std::size_t bytes) {
                  n = bytes;
              })
            | upon_error(
                  [&](std::error_code ec) {
                      // ec visible, n stale
                  }));
    }
}
```

The error code reaches `upon_error`. `when_all` would cancel siblings. `retry` would fire. The three channels are all in use.

What Approach A2 Loses

The byte count is not part of any completion signature. It is smuggled through a captured variable that the sender algorithms cannot see:

- `retry` fires on `set_error`, but the byte count is in `n`, not in the error channel. A generic retry algorithm cannot inspect the partial transfer count to decide whether retrying is safe.
- `upon_error` receives `error_code` alone. The byte count in `n` reflects the ***previous*** successful stage, not the failed one. The I/O sender called `set_error` without delivering the byte count to any channel.
- **Shared mutable state.** The byte count lives in a captured `std::size_t` that the programmer must coordinate across continuation boundaries. Shared mutable state between sender stages is the concurrency hazard the sender model was designed to eliminate.

Approach A2 uses all three channels, but the byte count bypasses them. The composition algebra operates on the error code alone. The compound result is still split - one half in a channel, the other in a side variable. This is Approach A1 with the error code moved to `set_error` and the byte count moved to shared state.

---

7. Approach B: Route Through `set_error`

The I/O sender calls `set_value(size_t)` on success and `set_error(error_code)` on any non-zero status. Completion signatures: `set_value_t(std::size_t)` and `set_error_t(std::error_code)`.

P3552R3[2] specifies that `set_error` is converted to an exception via `AS-EXCEPT-PTR` ([exec.general] p8). For `error_code`, this is `make_exception_ptr(system_error(ec))`, rethrown in `await_resume`. There is no non-throwing path:

```
auto do_session(auto& sock, auto& buf)
    -> std::execution::task<void>
{
    try {
        for (;;)
        {
            auto n = co_await async_read(
                sock, net::buffer(buf));

            co_await async_write(
                sock, net::const_buffer(buf, n));
        }
    } catch (std::system_error const& e) {
        // ECONNRESET, EPIPE, EOF arrive here
    }
}
```

What Approach B Loses

- **Information.** The byte count is discarded on any error. A `write` that sends 500 of 1,000 bytes before `ECONNRESET` produces `set_error(ECONNRESET)`. The 500 is gone.
- **Non-throwing error handling.** Every `ECONNRESET` - a routine network event - requires `make_exception_ptr` plus `rethrow_exception`.
- **Visible error path.** The error hides in a `catch` block separated from the `co_await` site. The programmer cannot inspect the error code and the byte count at the same call site.

---

8. The Trade-Off

Each sender approach pays a cost the coroutine-native approach does not.

Property	A1 (`set_value`)	A2 (shared state)	B (`set_error`)	Corosio
Error code at call site	Yes	Yes	No (in `catch`)	Yes
Byte count at call site	Yes	Yes	No (discarded)	Yes
Partial write preserved	Yes	Yes	No	Yes

Property	A1 (`set_value`)	A2 (shared state)	B (`set_error`)	Corosio
Byte count in completion sig	Yes	No (side state)	No (discarded)	Yes (return value)
`when_all` cancels on I/O error	No	Yes	Yes	Yes (value-based)
Error handler reachable	No (`upon_error`)	Yes (`upon_error`)	Yes (`upon_error`)	Yes (`if (ec)`)
Retry on I/O error	No (`retry`)	Yes (`retry`)	Yes (`retry`)	Yes (`retry_after`)
Retry sees byte count	No	No	No	Yes
Exception on `ECONNRESET`	No	No	Yes	No
Shared mutable state required	No	Yes	No	No
Channels used for I/O	1 of 3	3 of 3	3 of 3	Values (no channels)

Infrastructure operations (Section 3) face no such trade-off. Their outcomes are binary. The three channels map unambiguously.

The echo server is deliberately minimal. The compound-result problem is per-operation: every `read` returns `(status, bytes_transferred)` regardless of the protocol built on top of it. A TLS handshake, an HTTP/2 stream multiplexer, and a database wire protocol all issue reads and writes with the same completion shape. Adding protocol complexity adds more call sites with the same trade-off, not a different trade-off. The echo server isolates the structural problem. A more complex example would demonstrate the same table at every I/O boundary, with additional protocol logic between them. An HTTP/2 multiplexer issuing concurrent reads on multiple streams via `when_all` faces the same per-stream choice: Approach A1 preserves the byte count but `when_all` does not cancel siblings on I/O failure; Approach B cancels siblings but discards the byte count and throws on `ECONNRESET` .

---

9. How Coroutines Achieve Structured Concurrency

Coroutines alone are suspendable functions. They do not compose. The structure comes from the execution context - the event loop, the thread pool, the io_uring submission queue. The execution context collects suspended coroutines, dispatches completions, propagates stop tokens, and enforces lifetime boundaries. `when_all` launches coroutines into the execution context, maintains a completion counter, and requests

cancellation of siblings via stop tokens when the first coroutine completes. Each cancelled sibling decrements the counter as it completes; `when_all` resumes the caller when the counter reaches zero. The compound result is visible to the application before the cancellation decision is made. The classification happens with full protocol context, not at a context-free channel router.

The sender model provides structured concurrency through the type system: completion signatures, channel dispatch, and compile-time composition. The coroutine-native model provides structured concurrency through the runtime: the execution context manages the same concerns at the point where the application has already inspected the result. Both models provide structured concurrency. They differ in where the compound result is visible when the composition decision is made.

---

10. Invitation

The authors invite any reader to construct a sender-based echo server that:

- preserves compound I/O results (error code and byte count visible at the call site),
- retains channel-based composition (`when_all` , `upon_error` fire on I/O errors),
- avoids exception round-trips for routine error codes, and
- uses only facilities in C++26 or in-progress proposals (P3552R3[2], P3149[6]).

The authors will incorporate any such construction into a future revision and re-evaluate every finding in this paper. If the sender approach, when done correctly, matches the coroutine-native ergonomics, this paper will say so.

---

11. Acknowledgments

The authors thank Dietmar Kühl for the channel-routing enumeration in P2762R2[10] and for `beman::execution` , Chris Kohlhoff for identifying the partial-success problem in P2430R0[5], Kirk Shoop for the completion-token heuristic analysis in P2471R1[11], Peter Dimov for the refined channel mapping in P4007R0[9], Michał Dominiak, Eric Niebler, and Lewis Baker for `std::execution` , Ian Petersen, Jessica Wong, and Kirk Shoop for `async_scope` , Fabio Fracassi for P3570R2[12], Ville Voutilainen and Jens Maurer for reflector discussion, and Herb Sutter for identifying the need for constructed comparisons.

References

1. P2300R10 - ‘‘std::execution’’ (Michał Dominiak et al., 2024). https://wg21.link/p2300r10

2. P3552R3 - ‘‘Add a Coroutine Task Type’’ (Dietmar Kühl, Maikel Nadolski, 2025). https://wg21.link/p3552r3

3. cppalliance/corosio - Coroutine-native networking library. Commit ce1c43e. https://github.com/cppalliance/corosio/tree/ce1c43e623fb7b0e198ffac52be9267eccf04ecb

4. cppalliance/capy - Coroutine I/O primitives library. https://github.com/cppalliance/capy

5. P2430R0 - ‘‘Partial success scenarios with P2300’’ (Chris Kohlhoff, 2021). https://wg21.link/p2430r0

6. P3149 - ‘‘async_scope - Creating scopes for non-sequential concurrency’’ (Ian Petersen, Jessica Wong, Kirk Shoop, et al., 2024). https://wg21.link/p3149

7. D4054R0 - ‘‘Two Error Models’’ (Vinnie Falco, 2026). https://wg21.link/d4054r0

8. IEEE Std 1003.1 - POSIX `read()` / `write()` specification. https://pubs.opengroup.org/onlinepubs/9799919799/

9. P4007R0 - ‘‘Senders and Coroutines’’ (Vinnie Falco, Mungo Gill, 2026). https://wg21.link/p4007r0

10. P2762R2 - ‘‘Sender/Receiver Interface For Networking’’ (Dietmar Kühl, 2023). https://wg21.link/p2762r2

11. P2471R1 - ‘‘NetTS, ASIO and Sender Library Design Comparison’’ (Kirk Shoop, 2021). https://wg21.link/p2471r1

12. P3570R2 - ‘‘Optional variants in sender/receiver’’ (Fabio Fracassi, 2025). https://wg21.link/p3570r2